การสร้างและประเมินโมเดลสุขภาพหัวใจ

Workshop 5 Modeling and Evaluation ต่อจากข้อมูล Workshop 4

เลือกใช้ 5 เทคนิคทั่วไปตามสไลด์และขอบเขตที่อนุมัติ Classroom ไม่มีรายละเอียดจำนวนโมเดลเฉพาะ จึงไม่อ้างว่าอาจารย์ยืนยัน 5 เทคนิค ผลหลักมาจาก AI Studio native engine ผ่าน official batch แยกจาก Python reference ภาพ GUI และ Turbo Prep ยังรอหลักฐานจริง

เป้าหมายและ Inputs

HeartDisease เป็น label สองค่า โดย Yes เป็น positive class ใช้ตัวแปรเดิม 17 ตัวและ AnyUnhealthyDays รวม 18 inputs ไม่ใช้ Target หรือ SplitSet เป็น inputs และไม่ใส่ SourceRow/hash เป็นตัวแปรทำนาย งานเป็นการเรียนจำแนกคำตอบประวัติในไฟล์ ไม่ใช่การประมาณโอกาสเกิดโรคอนาคต

การแบ่งข้อมูลและป้องกันข้อมูลรั่ว

จัดกลุ่มด้วยค่าของ 17 predictors เดิมทั้งหมด โดยไม่ใช้ label เป็นตัวกำหนด profile แล้วใช้ StratifiedGroupKFold 10 folds, shuffle seed 2026 เลือก folds 0–2 เป็น test และ 3–9 เป็น train ทำให้ profile เดียวกันไม่ปรากฏข้ามชุดโดยยังเก็บทุกผู้ตอบ

รายการ	Train	Test
ระเบียน	223,855	95,940
HeartDisease Yes	19,161	8,212
สัดส่วน Yes	8.5596%	8.5595%
profile ข้ามชุด	0	0

ทุกโมเดลใช้ split เดียวกันตรงกับ SplitSet ใน heart_2020_modeling.csv และ split_assignment.csv Encoding กับ scaling fit เฉพาะ train แล้ว apply test ทดสอบค่า BMI สุดโต่งและหมวดหมู่ที่เกิดเฉพาะ held-out ว่าไม่ถูกเรียนรู้ใน training parameters แล้วคืนข้อมูลก่อนรันจริง

สัดส่วนระเบียนใน cleaned extract เป็นสถิติของไฟล์นี้ ไม่มี survey weights ในชุดคอลัมน์นี้ จึงไม่ตีความ 8.56% เป็นความชุกของประชากรสหรัฐ จาก CDC BRFSS ต้นฉบับมีวิธีสุ่มและ weighting ที่ต้องใช้อย่างเหมาะสมเมื่อประมาณระดับประชากร

ผลหลักจาก native AI Studio official batch

รันกระบวนการจริงด้วย ai-studio-batch.bat ใน AI Studio 2026.1.1 ใช้ training 223,855 และ test 95,940 แถวตาม fixed profile split เดียวกันทุกโมเดล positive class คือ HeartDisease Yes การตรวจ metrics ใช้ confusion counts ของ native exports

โมเดล	Accuracy	Precision Yes	Recall Yes	F1 Yes	ROC AUC
Naive Bayes	0.7542	0.2164	0.7140	0.3321	0.8120
Logistic Regression	0.8725	0.3307	0.4780	0.3909	0.8384
Decision Tree	0.9150	0.5524	0.0385	0.0719	0.5684
Random Forest	0.9154	0.6600	0.0241	0.0465	0.8255
Gradient Boosted Trees	0.8612	0.3126	0.5183	0.3900	0.8365

โมเดล	TN	FP	FN	TP
Naive Bayes	66,498	21,230	2,349	5,863
Logistic Regression	79,784	7,944	4,287	3,925
Decision Tree	87,472	256	7,896	316
Random Forest	87,626	102	8,014	198
Gradient Boosted Trees	78,369	9,359	3,956	4,256

เมือใช้ F1 Yes เป็นเกณฑ์ Logistic Regression สูงสุด 0.3909 โดย Precision 0.3307 และ Recall 0.4780 พลาด Yes 4,287 แถว จึงต้องอ่านความผิดพลาดร่วมกับ Accuracy และไม่ตีความเป็นการทำนายโรคในอนาคต

F1 ของสองอันดับแรกต่างกัน 0.0009 จึงไม่ใช่หลักฐานความต่างอย่างมีนัยสำคัญ ส่วน Random Forest มี Accuracy 0.9154 แต่ Recall 0.0241 และ FN 8,014 แสดงว่าการเลือกจาก Accuracy เพียงค่าเดียวอาจมองข้ามความผิดพลาดของกลุ่ม Yes

ครั้งแรกเกิด Java heap OOM ที Random Forest แบบ parallel จึงปิด enable_parallel_execution ตามคำแนะนำของ installed operator manual แล้วรันใหม่ด้วย heap 3 GB คง 100 trees, depth 12, leaf 10, seed 2026 และข้อมูลทุกแถวเดิม ไม่ลดขนาด train/test ตารางนีใช้ exports ใหม่จากการรันทีสำเร็จครบห้าโมเดลเท่านัน

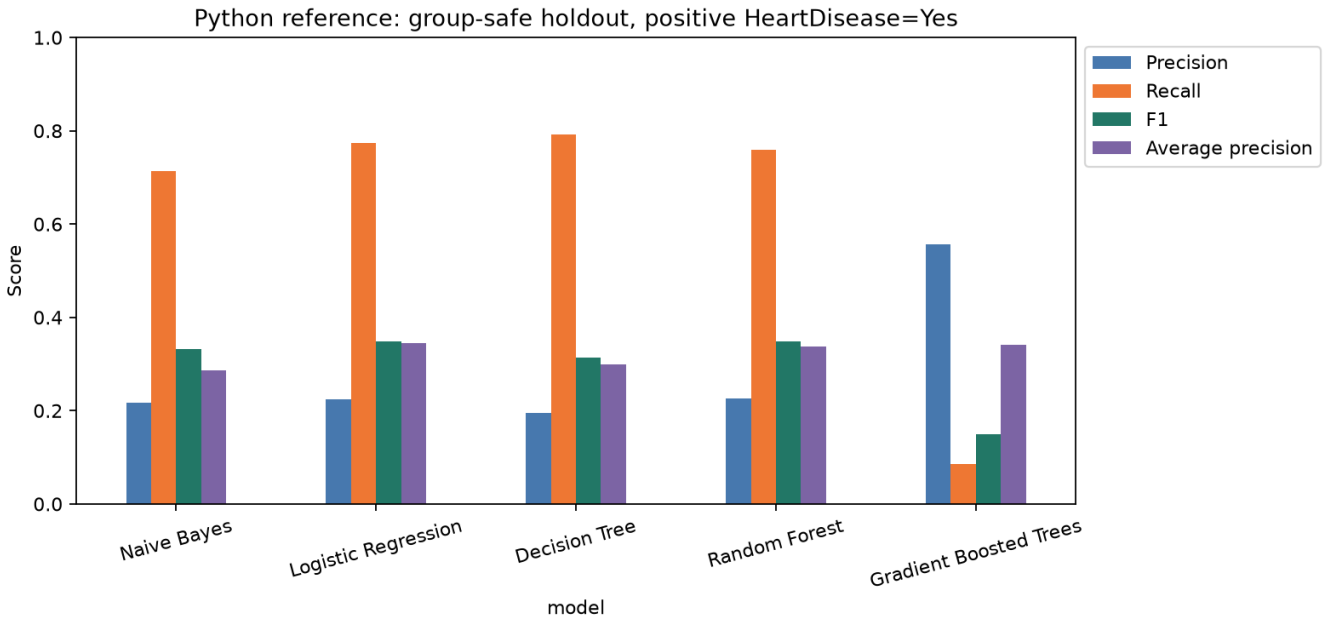
โมเดลและผล Python reference

ใช้ scikit-learn จาก runtime ทีเตรียมไว้ positive Yes, threshold 0.5 ไม่ปรับ hyperparameter หรือ threshold จาก test นี เลือกทังห้าเพือเปรียบเทียบพฤติกรรมของโมเดล ไม่ใช่ผล Auto Model ทีเก็บจาก GUI

โมเดล	ค่าตังต้น Python reference
Naive Bayes	GaussianNB var_smoothing=1e−9
Logistic Regression	C=1 balanced weights max_iter=1500
Decision Tree	depth=8 leaf=30 balanced weights
Random Forest	100 trees depth=12 leaf=10 balanced weights
Gradient Boosted Trees	60 trees depth=3 leaf=20 learning_rate=0.1

โมเดล	Accuracy	Precision Yes	Recall Yes	F1 Yes	Average Precision
Naive Bayes	0.7542	0.2164	0.7140	0.3321	0.2871
Logistic Regression	0.7513	0.2241	0.7742	0.3476	0.3444
Decision Tree	0.7032	0.1955	0.7921	0.3136	0.2991
Random Forest	0.7563	0.2255	0.7589	0.3477	0.3382
Gradient Boosted Trees	0.9159	0.5566	0.0862	0.1493	0.3408

Baseline ทาย No ทังหมดได้ Accuracy 0.9144 แต่ Recall และ F1 ของ Yes เป็น 0 ค่า Average Precision ของการจัดอันดับสุ่มมีฐานประมาณ 0.0856. Class imbalance จึงต้องอ่าน Precision/Recall/F1 และความผิดพลาดร่วมกับ Accuracy



รูปจากผล Python reference เท่านัน ไม่ใช่ภาพ Results ของ AI Studio

การประเมินและข้อผิดพลาดของ Python reference

โมเดล	TN	FP	FN	TP
Naive Bayes	66,495	21,233	2,349	5,863
Logistic Regression	65,719	22,009	1,854	6,358
Decision Tree	60,963	26,765	1,707	6,505
Random Forest	66,327	21,401	1,980	6,232
Gradient Boosted Trees	87,164	564	7,504	708

TN คือทาย No ถูก FP คือทาย Yes แต่จริง No FN คือพลาด Yes และ TP คือทาย Yes ถูก ทุกแถวในตารางรวมเป็น 95,940 test rows และตรวจคำนวณ metrics กลับจาก confusion counts ได้

Random Forest มี F1 0.3477 และ Logistic Regression 0.3476 ต่างกันประมาณ 0.0001
จึงยังไม่มีหลักฐานเพียงพอให้เลือกจากช่อง F1 นีเพียงอย่างเดียว Logistic Regression มี Average Precision 0.3444 และ ROC AUC 0.8387 สูงสุดใน reference ชุดนี พร้อมโมเดลทีอธิบายสัมประสิทธิได้
จึงเป็นตัวเลือกเพือศึกษาต่อภายใต้เกณฑ์เชิงการเรียนรู้

อย่างไรก็ตาม Logistic Regression มี FP 22,009 และ Precision 0.2241 จึงเห็นผลของการใช้ balanced class weights และความไม่สมดุล ส่วน Gradient Boosted Trees มี Accuracy 0.9159 และ Precision 0.5566 แต่ Recall เพียง 0.0862 พลาด Yes 7,504 แถว จึงไม่ควรเลือกจาก Accuracy สูงสุดเพียงค่าเดียว

Decision Tree ให้ Recall 0.7921 สูงสุด แต่มี FP 26,765 ขณะที Naive Bayes ให้ Recall 0.7140 และ F1 0.3321 ผลทังหมดเป็น association ในคำตอบย้อนหลัง ไม่มีการทดสอบภายนอกหรือข้อมูลในอนาคตเพือยืนยันการใช้งานอืน

การนำผลไปใช้ต่อ

สำหรับ Workshop ใช้ความต่างของโมเดลอธิบาย tradeoff และตรวจผลจากโปรแกรมจริง ขันต่อไปควรเลือก threshold ด้วย validation ทีแยกจาก test, ประเมินความคงทีของผล และทบทวนความหมายของ features ทีเกียวกับโรคเดิม เป้าหมายคืออธิบายสิงทีโมเดลเรียนได้จากข้อมูลนีก่อนขยายขอบเขต

กระบวนการและหลักฐาน native AI Studio

Workshop5_Modeling_Evaluation.rmp อ่าน heart_2020_modeling.csv และกำหนด HeartDisease label ใช้ Filter Examples แบ่ง SplitSet=train ออกทาง matched ส่วน test ออกทาง unmatched จากนันตัด SplitSet ก่อน training-only encoding/scaling และใช้ชุดเดียวกันในห้าแขนง

ใช้ Naive Bayes core, H2O Logistic Regression, parallel Decision Tree, parallel Random Forest และ H2O Gradient Boosted Trees ทีตรวจ operator IDs/ports จากไฟล์ติดตังจริง Python และ native implementations/weights/defaults ต่างกัน จึงต้องบันทึกผล native แยก ไม่อ้างว่าผลจะเท่าตาราง reference

พอร์ต Result 1–5 คือ Performance, 6–10 คือ scored test, 11–15 คือ models และ 16–20 คือ metric tables หลัง Run จริง Performance to Data ส่งออก native_NB.csv, native_LR.csv, native_DT.csv, native_RF.csv และ native_GBT.csv ผ่าน Write CSV โดยไม่ต้องถอดตัวเลขจากภาพ

ต้องเปิด .rmp แบบไฟล์ในเครืองพร้อม original heart_2020_cleaned.csv และ derived heart_2020_modeling.csv ในโฟลเดอร์เดียวกัน ตรวจ macros ทีอ้าง process_path/process_file แล้ว Run ทังกระบวนการ เก็บภาพ setup, process และ performance/confusion matrix ทังห้า จากนันย้ายทังโฟลเดอร์และเปิด Run ใหม่

สถานะ: native batch รันสำเร็จครบห้าโมเดล ส่งออก native_NB/LR/DT/RF/GBT.csv และผ่านการตรวจ confusion reconciliation จำนวน test 95,940 และ positive Yes 8,212 ทุกโมเดล รวม hash/เวลา exports ใน native_metrics_provenance.json ผล Python reference เก็บแยก การย้ายสำเนาไปโฟลเดอร์ใหม่และ native batch รันซ้ำผ่านแล้ว ผลห้าโมเดลตรงกับการรันหลัก ดู docs/execution/native-portability-verification.json ภาพ GUI และ Turbo Prep ยังรอหลักฐานจริง

แหล่งข้อมูลและบทเรียน

Kamil Pytlak Indicators of Heart Disease Kaggle https://www.kaggle.com/datasets/kamilpytlak/personal-key-indicators-of-heart-disease ใช้ heart_2020_cleaned.csv จำนวน 319,795 ระเบียน ไม่เปลียนเป็นไฟล์ปี 2022 ตามชือหน้าปัจจุบัน

CDC BRFSS 2020 https://www.cdc.gov/brfss/annual_data/annual_2020.html และ Summary Matrix of Calculated Variables https://www.cdc.gov/brfss/annual_data/2020/Summary_Matrix_20.html นิยามคำตอบประวัติโรคหัวใจมาจาก coronary heart disease หรือ myocardial infarction

บทเรียน BI07 Data Model หน้า PDF 4 สไลด์ 13–15 เรือง train/test หน้า 12 สไลด์ 45–46 เรือง imbalance และหน้า 15 สไลด์ 57–60 เรือง Logistic Regression และขอบเขตทีอนุมัติให้ใช้ข้อมูลทีอาจารย์เคยโพสต์ Classroom ไม่ได้ให้คำสังไฟล์หรือจำนวนโมเดลเฉพาะงานนี